PROMPT LIFE V0.9.1 · BATCH 2 · JUNE 3, 2026

Inference to Tensors

Implemented Batch 2 of the curriculum and fixed the Batch 1 visual/PDF layout issues with HTML legends, numbered callouts, and print-safe captions. Final app polish is visible as v0.9.1.

What Changed

- Implemented full cards for Inference, Prompt vs Response, Tokenization, Token IDs, Embeddings, Vectors, and Tensors.
- Bumped the visible app version to `v0.9.0`.
- Applied a v0.9.1 polish fix for duplicate React keys in repeated Prompt vs Response token cards.
- Moved lessons 8-14 into the complete lesson schema: definition, core explanation, where it happens, durable/temp, prompt/response, metaphor, Brain Bridge, misconception, visual aid, and checkpoint.
- Improved Batch 1 visuals for Rationalists vs Empiricists, Training, Overfitting, and Alignment.
- Updated the glossary for tokenizer, tokenization, embedding table, distributed representation, activation, hidden state, and weight tensor.
- Updated the default lesson-card PDF path to `docs/review/prompt-life-lesson-cards-v0-9.pdf`.
- Generated the v0.9 lesson-card PDF at `docs/content-inventory/prompt-life-lesson-cards-v0-9-batch-2.pdf`.

How To Run Locally

From this repo:

```
cd ~/Desktop/promptlife
npm install
npm run dev
```

Open `http://localhost:5173/`. For same-Wi-Fi phone testing, run `npm run dev -- --host 0.0.0.0` and open the Network URL Vite prints.

Verification

- `npm install`: passed.
- `npm run build`: passed.
- `npm run export:lesson-cards`: passed.
- Mobile overflow audit: passed at 320px, 390px, and 430px.
- PDF sample page: no clipped visual, vertical caption, or unreadable label found.
- `npm run typecheck`: passed.
- `npm run export:lesson-pdf`: passed.
- GitHub Pages deploy from `main`: passed.
- Review route: 28 lesson cards, Batch 2 present.
- No new games, Three.js, or heavy 3D library added.

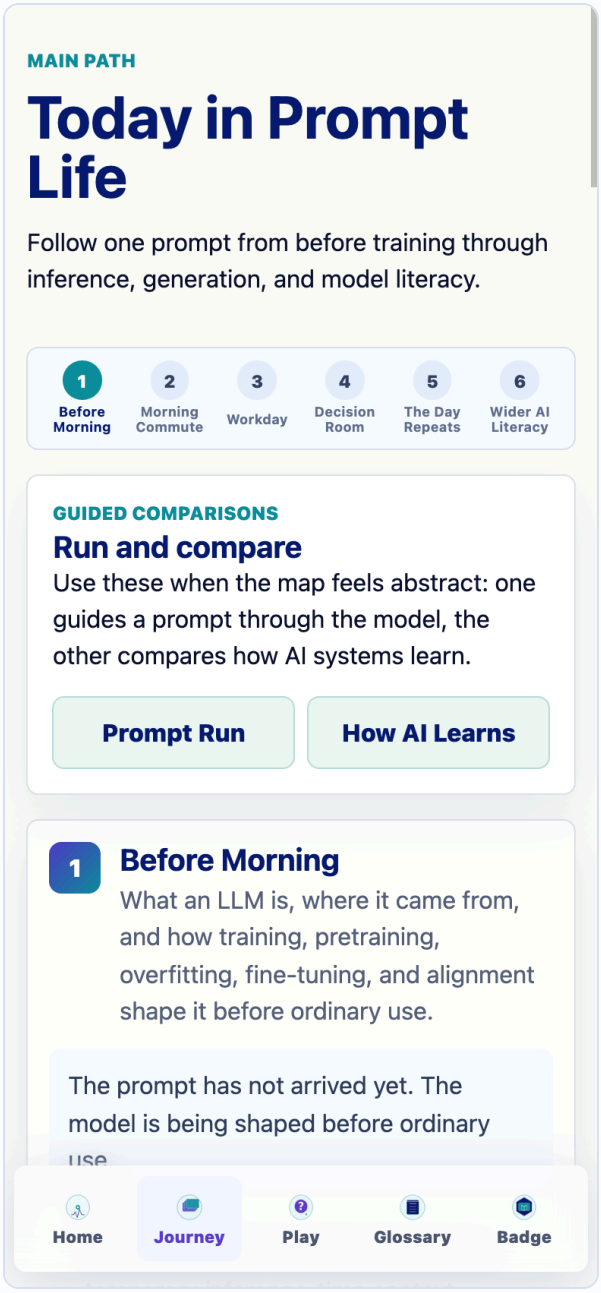
Files Changed

`README.md`, `package.json`, `scripts/export-lesson-pdf.mjs`, `src/components/VisualAids.tsx`, `src/data/content.ts`, `src/main.tsx`, `src/styles/global.css`, `docs/REVIEW_NOTES.md`, Batch 2 curriculum docs, v0.9 lesson-card PDFs, and v0.9 screenshot artifacts.

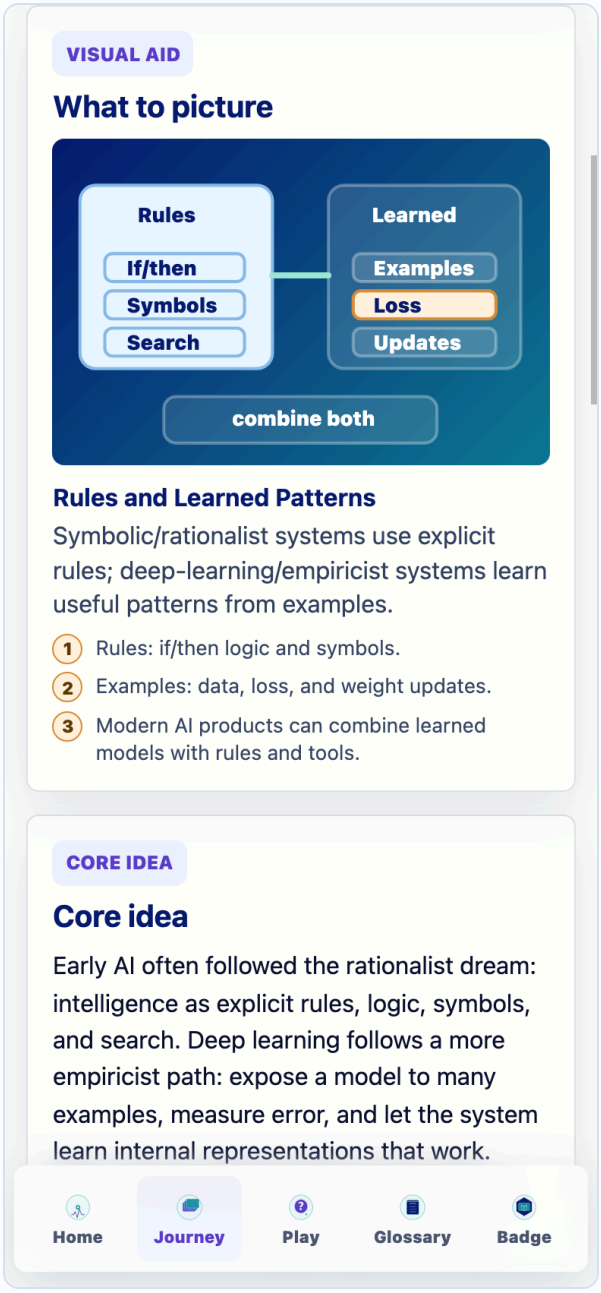
Next Three v0.10 Improvements

1. Implement Batch 3 for Attention, MLP, Layers, and Hidden States as a connected transformer-workday section.
2. Add source citations or a bibliography layer for Batch 1 and Batch 2 before broader publication.
3. Add an automated screenshot/overflow script to the repo so future visual-aid changes can be checked repeatably.

Screenshots



Journey top



Batch 1 fixed: Rationalists vs Empiricists

VISUAL AID

What to picture

Training Loop

Predict, compare, loss, update weights, repeat. Training changes weights.

1. Predict and compare against a target.
2. Loss measures error.
3. Weight updates make training durable.

CORE IDEA

Core idea

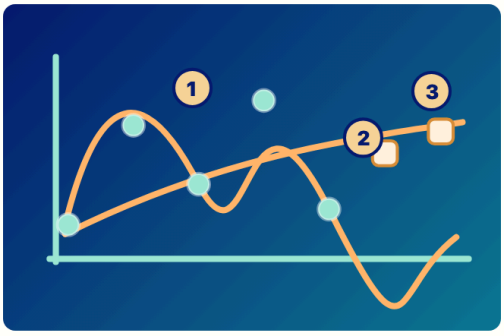
During training, the model sees examples, predicts something, measures how wrong it was, and updates its weights to do slightly better next time. This happens over many examples and many updates. Training is where durable model capability is created.

MODEL LIFECYCLE

Batch 1 fixed: Training Loop

VISUAL AID

What to picture



Overfitting vs Generalization

Memorizing examples is not the same as learning transferable patterns.

1. Overfit curve traces old dots too tightly.
2. Generalizing curve is smoother and reaches new examples.
3. Held-out examples test whether learning transfers.

CORE IDEA

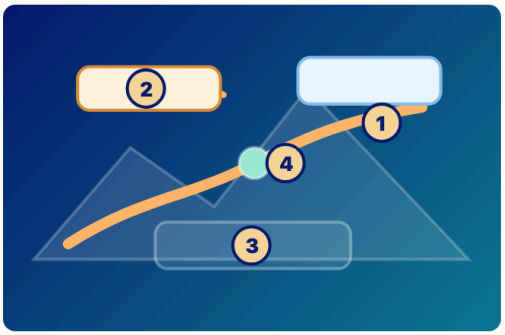
Core idea

A model can fit old examples very well but still fail on new cases. That is overfitting. Good learning captures patterns that transfer. For LLMs, generalization is why a model can answer prompts it has never seen exactly before. Overfitting is why evaluation

Batch 1 fixed: Overfitting

VISUAL AID

What to picture



Alignment Landscape

Alignment shapes behavior with preferred paths, guardrails, feedback, and policies; it does not create moral agency.

1. Preferred path: behavior the system encourages.
2. Warning zone: outputs to avoid or handle carefully.
3. Policy check: runtime or review constraint.
4. Feedback: behavior signal, not conscience.

CORE IDEA

Core idea

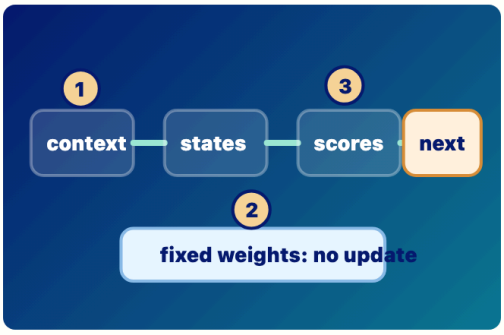
A model trained only to predict likely text may produce outputs that are fluent but unhelpful, unsafe, biased, or misleading. Alignment uses methods such as instruction

Home Journey Play Glossary Badge

Batch 1 fixed: Alignment

VISUAL AID

What to picture



Forward Pass

Inference creates temporary states while durable weights stay fixed.

1. Current context enters the model.
2. Fixed weights are used, not updated.
3. Temporary hidden states lead to next-token scores.

CORE IDEA

Core idea

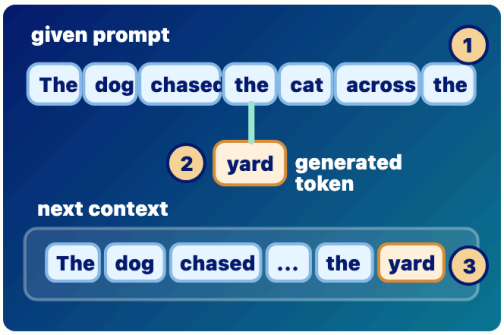
During ordinary use, the current context enters the model, embeddings and hidden states are computed, the final hidden state produces next-token scores, and decoding chooses one response token. Then the token is appended and the process repeats. The learned weights are used, but they normally

Home Journey Play Glossary Badge

Batch 2: Inference

VISUAL AID

What to picture



Prompt vs Response

Prompt tokens are given; response tokens are generated and appended.

1. Prompt tokens are supplied context.
2. The generated token yard is appended.
3. The next run sees prompt plus response so far.

CORE IDEA

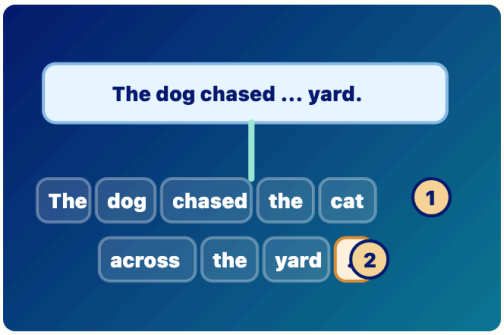
Core idea

The model does not process the prompt once and then write a whole answer. It repeatedly runs on the current context. At first, the context contains the prompt and any prior conversation or retrieved material. After the model chooses one response token, that token is appended. The next forward pass sees the prompt plus the

Batch 2: Prompt vs Response

VISUAL AID

What to picture



Text to Tokens

Text is split into model-readable chunks before embedding lookup.

1. Tokens can be words, word pieces, punctuation, or spaces.
2. This app uses simplified tokens; real tokenizers may split differently.

CORE IDEA

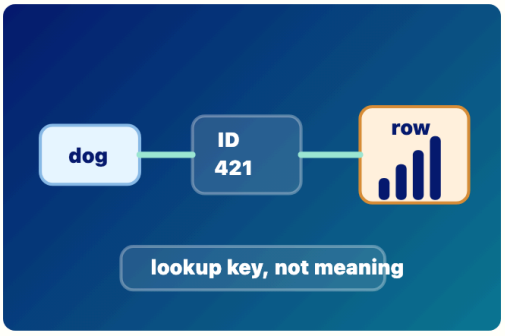
Core idea

LLMs do not carry raw English sentences through their layers. Text is first split into tokens: pieces that might be whole words, word parts, punctuation, or spaces. Prompt text is tokenized before the model processes it. Generated response tokens are also token IDs before they are turned back into visible

Batch 2: Tokenization

VISUAL AID

What to picture



Token IDs

Each token gets a lookup number that points to an embedding row.

1. The token is not carried forward as raw English.
2. The ID is a lookup key, not the meaning.
3. The ID points to an embedding row.

CORE IDEA

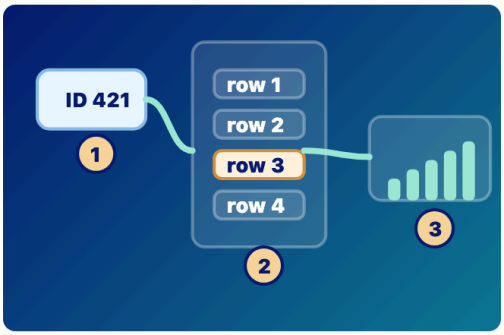
Core idea

After tokenization, each token is represented by an integer ID. The ID is not the meaning. It is a lookup key. The embedding table uses that ID to retrieve the token's learned starting vector. During generation, the model also chooses a next token ID before it is displayed as text.

Batch 2: Token IDs

VISUAL AID

What to picture



Embedding Lookup

A token ID retrieves a learned starting vector.

1. The embedding table was learned during training.
2. Inference retrieves one row temporarily for the current context.
3. Later layers reshape embeddings into hidden states.

CORE IDEA

Core idea

The embedding table is learned during training. At inference time, each token ID in the current context retrieves one row from that table. That row is the token's starting vector before context changes it. Later layers turn embeddings into hidden states

Batch 2: Embeddings

VISUAL AID

What to picture

Feature Vector

A vector is a list of numerical features, not a sentence.

1. Slider labels are simplified teaching examples.
2. Real features are distributed across many dimensions.

CORE IDEA

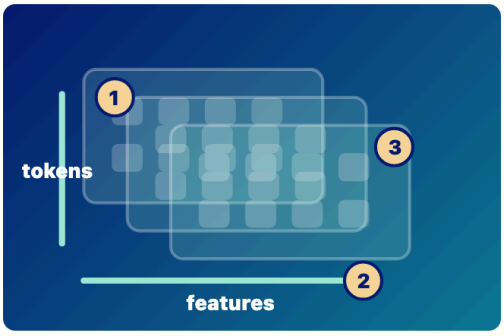
Core idea

In an LLM, many things are represented as vectors. An embedding is a starting vector for a token ID. A hidden state is a later vector shaped by context. The numbers are not usually neat labeled features like dogness or grammar. Features are distributed across many dimensions, which is why vectors can represent fuzzy relationships.

Batch 2: Vectors

VISUAL AID

What to picture



Tensor Block

Tensors organize many token vectors for layer-by-layer processing.

1. Token positions run down one axis.
2. Feature dimensions run across another axis.
3. Batch x tokens x features is an advanced shape note.

CORE IDEA

Core idea

Once tokens have embeddings, the model needs to carry many token positions and many features together. A simple way to picture this is a table: token positions down one side, feature numbers across the other. With batches, heads, and layers, the shapes get richer. But the core idea is simple:

Batch 2: Tensors

PROMPT LIFE V0.9.1

Lesson Cards

One accessible curriculum-review card per lesson, built from current app data plus inventory notes and rubric scores.

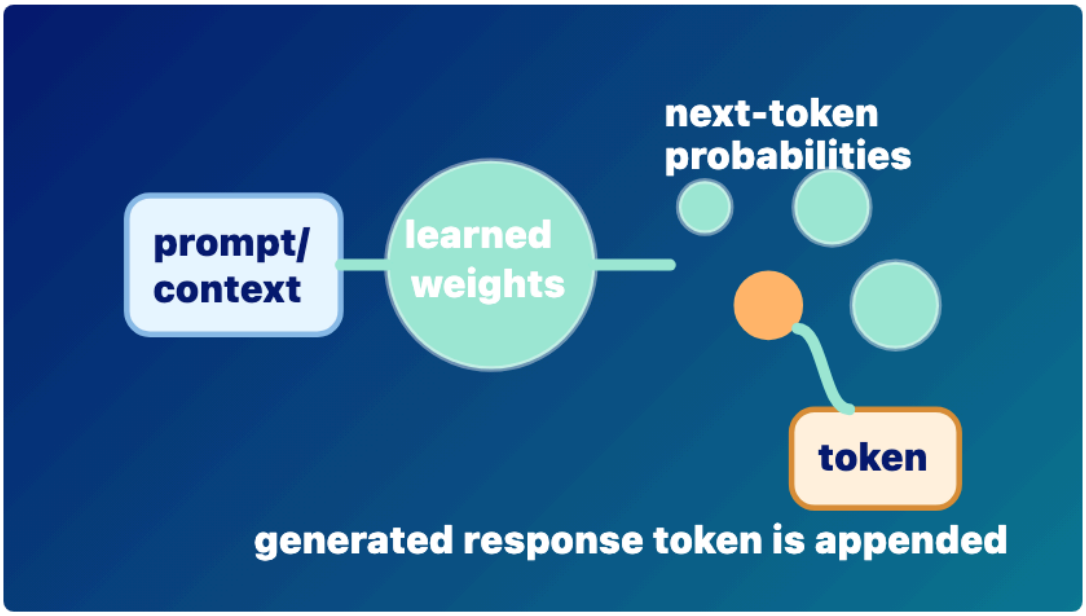
1 / 28

BEFORE MORNING

What Is an LLM?

A learned next-token prediction system

Stage: architecture | Priority: low | Rubric avg: 4.4/5



Prompt to Prediction

Prompt/context flows through learned weights into next-token probabilities; one generated response token is appended.

1. Prompt/context is given input.
2. Learned weights are used during inference.
3. The selected response token is appended.

DEFINITION

A large language model is a learned system that turns current context into probabilities for the next token.

CORE EXPLANATION

An LLM does not read a prompt and write a whole answer at once. During inference, the current context enters the model, learned weights shape internal hidden states, the model scores possible next tokens, one token is chosen, and that token is appended before the next run. Training built the weights earlier; inference uses them.

WHERE IT HAPPENS

This card describes the whole model lifecycle, but especially normal inference: current context to next-token probabilities to generated response token.

DURABLE VS TEMPORARY

The weights were changed during training or fine-tuning. During ordinary inference, hidden states are temporary and weights normally stay fixed.

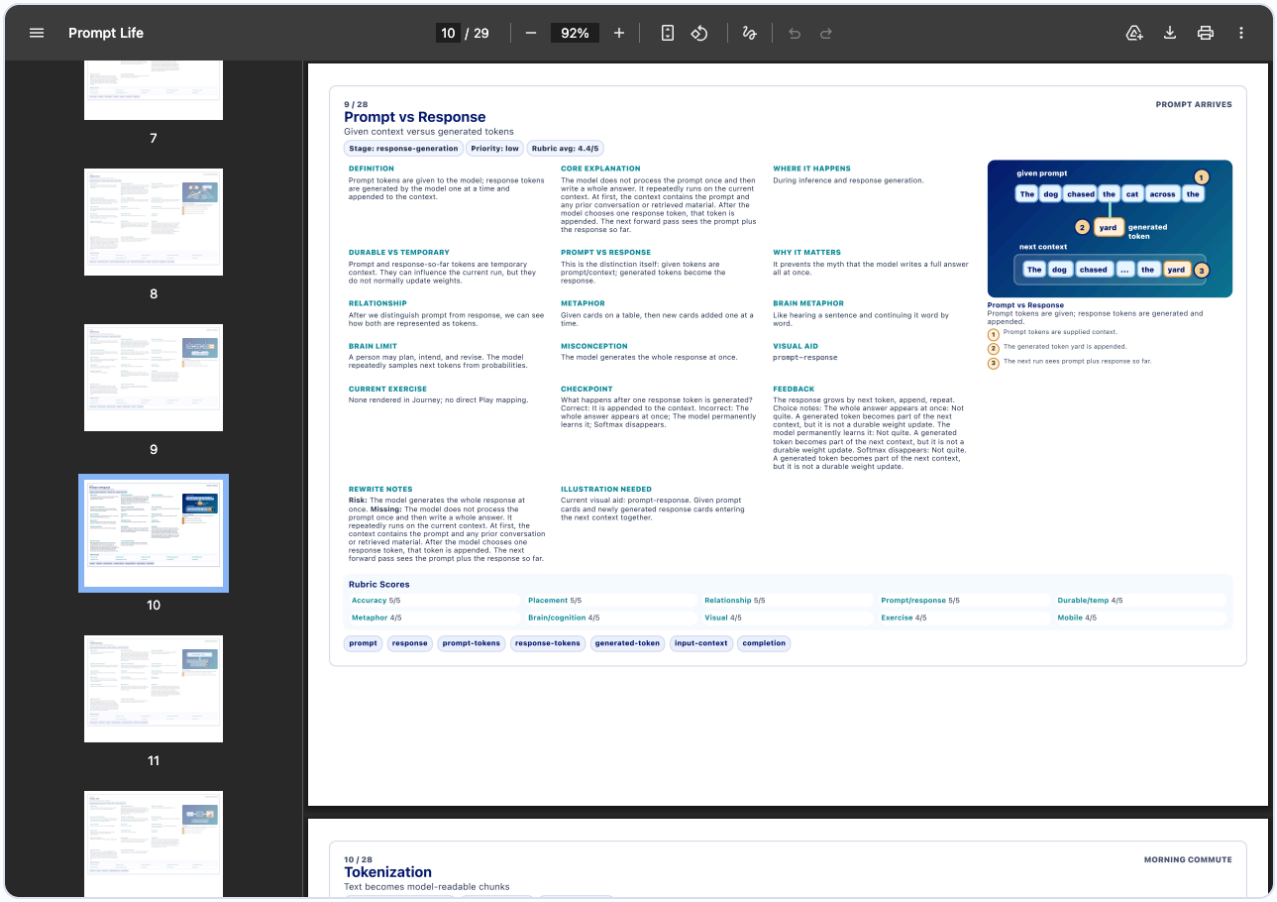
PROMPT VS RESPONSE

Prompt tokens are given to the model. Response

WHY IT MATTERS

This definition lowers the mystery. The model can be

Review route sample



PDF sample page